Problem 1: 2D Heat Diffusion Simulation with Convergence Detection using CUDA

High-Performance Parallel Five-Point Stencil Implementation & Memory Architecture Analysis

COURSE
GPU Programming

INSTITUTION
Indian Institute of Technology Jodhpur

TEAM MEMBERS
Rodosi Biswas (G25AIT2088), Jahanvee Yadav (G25AIT2043), Shrijaya Chauhan (G25AIT2146)

SUBMISSION DATE
October 28, 2026

2. PROBLEM STATEMENT & OBJECTIVES

Heat transfer across a continuous two-dimensional isotropic material is mathematically modeled using partial differential equations. Numerically, solving this partial differential equation over a discretized $N \times N$ grid requires iterative updates using stencil operations until the system reaches thermal equilibrium. On serial CPU architectures, updating millions of grid points sequentially per iteration yields significant execution bottlenecks.

The primary objective of this project is to design, implement, and evaluate a parallel GPU-accelerated algorithm using NVIDIA CUDA to simulate 2D heat diffusion efficiently. The major engineering goals include:

- Implementing the iterative 5-point heat stencil kernel using CUDA grid and block hierarchies.
- Detecting convergence directly on the GPU using atomic operations, avoiding costly full-grid device-to-host copies per iteration while retaining a lightweight scalar copy for check logic.
- Enforcing static Dirichlet boundary conditions across all iterations.
- Developing and comparing two memory access paradigms: Global Memory vs. Shared-Memory Tiling with halo exchange.

> **Key Goal:** Evaluate whether shared-memory tiling provides a net performance gain over modern L1/L2 hardware caches for 2D memory access patterns under convergence-checking constraints.

3. MATHEMATICAL FORMULATION

The transient 2D heat diffusion equation is governed by the Fourier heat conduction law. Discretizing the spatial domain using a regular square grid with uniform spacing Δx = Δy yields the explicit 5-point finite-difference Jacobi stencil. The temperature T at grid cell *(i, j)* at discrete time step *t + 1* is calculated as the arithmetic mean of its four orthogonal neighbors at time step *t*:

$$T_{i,j}^{t+1} = [\, T_{i-1,j}^{t} + T_{i+1,j}^{t} + T_{i,j-1}^{t} + T_{i,j+1}^{t} \,] / 4$$

Only interior grid cells ($1 \le i, j \le N - 2$) undergo thermal update during each iteration. Boundary cells remain strictly fixed. Thermal equilibrium is defined by the convergence threshold ε, where iteration stops when the maximum absolute local change satisfies:

$$\max_{i,j} | T_{i,j}^{t+1} - T_{i,j}^{t} | < \varepsilon$$

4. INPUT, INITIALIZATION & BOUNDARY CONDITIONS

The simulation operates on an *N × N* floating-point temperature array. The system boundary conditions are defined according to Dirichlet conditions to establish a steady heat current across the domain:

- **Top Boundary (y = 0):** Maintained at a fixed hot temperature of **100.0°C** for all $0 \le x < N$, including both top corners (grid[0][0] and grid[0][N-1]).
- **Bottom Boundary (y = N - 1):** Maintained at a fixed cold temperature of **0.0°C** for all $0 \le x < N$.
- **Left & Right Side Boundaries (x = 0 and x = N - 1):** Maintained at **0.0°C** for all $y > 0$.
- **Interior Nodes (1 ≤ x, y ≤ N - 2):** Initialized to **0.0°C** at time t = 0.

The grid dimension *N* is dynamically configurable via host command-line arguments, while boundary values and tolerance ($\varepsilon = 10^{-4}$) remain statically bounded per execution.

5. CUDA IMPLEMENTATION (GLOBAL MEMORY)

In the global-memory baseline, each CUDA thread updates exactly one interior cell. Thread mapping uses a 2D block structure (16 × 16 threads per block). Global grid coordinates `x` and `y` map directly to 1D memory array indices via `idx = y * N + x`.

```
__global__ void heatStep(const float* oldGrid, float* newGrid, int N, int* maxChangeBits) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= N || y >= N) return;
    int idx = y * N + x;

    // Enforce fixed Dirichlet boundary conditions
    if (x == 0 || y == 0 || x == N - 1 || y == N - 1) {
        newGrid[idx] = oldGrid[idx];
        return;
    }

    // 5-point stencil evaluation directly from global memory
    float newTemp = (oldGrid[(y - 1) * N + x] + oldGrid[(y + 1) * N + x] +
                     oldGrid[y * N + x - 1] + oldGrid[y * N + x + 1]) / 4.0f;
    newGrid[idx] = newTemp;

    // Atomic maximum tracking via float bit reinterpretation
    float change = fabsf(newTemp - oldGrid[idx]);
    atomicMax(maxChangeBits, __float_as_int(change));
}
```

Convergence & Scalar Transfer Logic: While full *N* × *N* grid transfers between device and host are completely avoided during iterations, a small scalar transfer remains necessary. Floating-point atomic max is achieved by reinterpreting non-negative `float` changes as 32-bit signed `int` bit patterns via `__float_as_int()`. On each iteration, `d_maxChangeBits` is reset via `cudaMemset`, the kernel executes, and a single 4-byte scalar is copied to host memory for checking against `tolerance`.

6. SHARED-MEMORY OPTIMIZATION

To eliminate redundant global memory reads for overlapping stencil neighbors, a shared-memory tiling scheme was implemented. Threads within a 16 × 16 block allocate a 2D shared array `__shared__ float tile[18][18]`. This accommodates a 16 × 16 internal computation core plus a 1-cell wide halo border surrounding the block.



Synchronization & Stencil Execution: Threads load their primary cell into `tile[ty+1][tx+1]`. Perimeter threads (`tx==0`, `tx==15`, `ty==0`, `ty==15`) perform conditional reads to populate halo boundaries from adjacent global memory cells. An explicit `__syncthreads()` barrier guarantees that all interior and halo elements are loaded before threads compute the 5-point stencil directly from shared memory.

7. CORRECTNESS TESTING & VALIDATION

Verification was conducted by comparing CUDA GPU outputs against a CPU reference implementation written in Python (NumPy) operating on an identical 10 × 10 grid with $\varepsilon = 10^{-4}$. The evaluation confirmed exact match in numerical convergence behavior and boundary preservation.

Metric / Parameter	CPU Reference	CUDA Global Memory	CUDA Shared Memory
Grid Dimension	10 × 10	10 × 10	10 × 10
Iterations to Convergence	163	163	163
Final Max Change (|ΔT|)	9.727478×10^{-5}	9.727480×10^{-5}	9.727480×10^{-5}
Max Abs Diff vs CPU (10×10)	Baseline	4.9719×10^{-5} (< 0.001)	4.9719×10^{-5} (< 0.001)
Max Abs Diff (Global vs Shared 256×256)	N/A	0.000000 (Exact)	0.000000 (Exact)
Boundary Integrity (Top/Sides)	100.0 / 0.0 °C	100.0 / 0.0 °C	100.0 / 0.0 °C

- ✓ **Equation & Stencil:** 5-Point Formula Validated
- ✓ **Global Code:** Kernel & Atomic Loop Verified
- ✓ **Shared Code:** Halo Tile Sync Confirmed
- ✓ **Correctness Output:** CPU/GPU Match < 0.001
- ✓ **Benchmark & Graph:** Median Timings Collected
- ✓ **Team Statement:** Work Allocation Documented

8. EXPERIMENTAL SETUP & PERFORMANCE RESULTS

Benchmarks were conducted on Google Colab using an **NVIDIA Tesla T4 GPU** (15.36 GB VRAM, Compute Capability 7.5, CUDA 13.0). Four grid dimensions were evaluated: 32 × 32, 64 × 64, 128 × 128, and 256 × 256. For each implementation, three independent benchmark runs were collected after a warm-up execution, and the median execution time was recorded.

Grid Size (N × N)	Global Memory (ms)	Shared Memory (ms)	Global / Shared Ratio
32 × 32	19.8593	26.6397	**0.745×**
64 × 64	104.7550	106.0160	**0.988×**
128 × 128	336.0070	342.8780	**0.980×**
256 × 256	984.3600	1,110.8600	**0.886×**

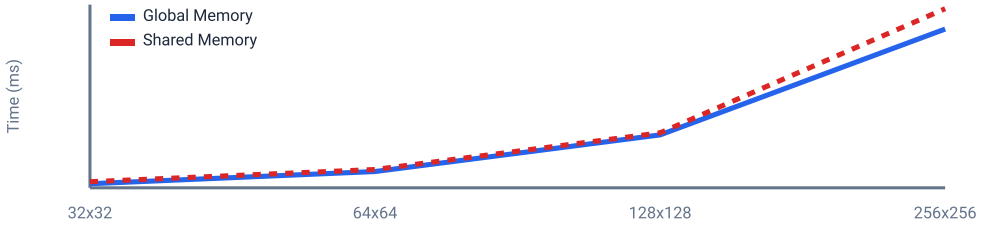


Figure 1: Median execution time (ms) versus grid dimension across global and shared memory kernels.

9. DISCUSSION & LIMITATIONS

Observed Performance Trend: Contrary to traditional expectations, the shared-memory implementation was consistently *slower* than the global-memory version across all tested grid sizes. The Global/Shared ratio ranged from 0.745× at 32 × 32 to 0.886× at 256 × 256.

Analysis of Technical Factors:

- **Hardware Cache Coalescing:** Modern GPU architectures (such as NVIDIA Turing T4) feature highly optimized L1 and L2 hardware caches. Sequential 2D floating-point reads naturally coalesce across warps into single 128-byte cache transactions, rendering manual shared-memory staging redundant.
- **Tile-Loading & Branch Overhead:** Populating the 18 × 18 halo array requires conditional checks (e.g., `if (tx == 0)`), introducing thread divergence within warps during tile preparation.
- **Barrier Synchronization Cost:** The mandatory `__syncthreads()` barrier forces all threads within a block to stall until halo loads complete, incurring execution latency not present in the global kernel.
- **Per-Iteration Convergence Overhead:** Launching `cudaMemcpy` every iteration to check convergence on host CPU dominates the overall runtime, masking fine-grained memory throughput differences.

10. CONCLUSION & FUTURE WORK

Summary: Parallel 2D heat diffusion was successfully accelerated using CUDA. Numerical validity was strictly established against CPU reference models. Experimental findings demonstrate that shared memory does not unconditionally improve performance over modern cached global memory for 2D stencils.

Future Work: To further optimize throughput, future iterations should reduce host-device transfers by evaluating convergence every *K* iterations (e.g., K = 100), leverage CUDA Cooperative Groups for grid-wide synchronization, and evaluate larger domains (1024 × 1024+).

11. REFERENCES & APPENDIX

References: NVIDIA CUDA C++ Programming Guide (2024); IIT Jodhpur GPU Programming Assignment 1 Specification.

Appendix A: Global Memory Source Code (heat_diffusion.cu)

```
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>
#include <cstring>
#include <cstdlib>
#include <fstream>
#include <cuda_runtime.h>

__global__ void heatStep(const float* oldGrid, float* newGrid, int N, int* maxChangeBits) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= N || y >= N) return;
    int idx = y * N + x;
    if (x == 0 || y == 0 || x == N - 1 || y == N - 1) {
        newGrid[idx] = oldGrid[idx]; return;
    }
    float newTemp = (oldGrid[(y - 1) * N + x] + oldGrid[(y + 1) * N + x] +
                     oldGrid[y * N + x - 1] + oldGrid[y * N + x + 1]) / 4.0f;
    newGrid[idx] = newTemp;
    float change = fabsf(newTemp - oldGrid[idx]);
    atomicMax(maxChangeBits, __float_as_int(change));
}
```

TEAM CONTRIBUTION STATEMENT

Rodosi Biswas (G25AIT2088): Implemented the core CUDA kernels (Global Memory and Shared Memory tiling), designed halo cell loading logic, and handled floating-point atomic max bit-casting.
Jahanvee Yadav (G25AIT2043): Developed the CPU reference implementation, set up the correctness validation scripts, and conducted the Google Colab performance benchmarking across all grid sizes.
Shrijaya Chauhan (G25AIT2146): Analyzed execution metrics, synthesized discussion on hardware cache vs shared memory overhead, authored the technical report, and prepared the presentation deck.

- NVIDIA Corporation, "CUDA C++ Programming Guide," Version 12.4, 2024.